Introduction to Python for Statistical Data Analysis

Ms. Beryl Ang'iro

Course:	STA 326: Statistical Computing II
Department:	Department of Mathematics, Statistics & Actuarial Science
Language:	Introduction to Python for Statistical Data Analysis

These notes are designed for students with **no prior programming experience**.

By the end of these notes, you will be able to write basic Python programs, understand fundamental programming concepts, and have the foundation to explore more advanced topics.

"Python is an easy to learn, powerful programming language."
Beryl Ang'iro

Contents

What is Python?

Python is a **high-level, general-purpose programming language** created by Guido van Rossum and first released in 1991. It is one of the most popular programming languages in the world today, widely used in:

- **Data Science and Machine Learning** — analysing large datasets, building AI models
- **Web Development** — building websites and web applications
- **Automation** — automating repetitive tasks
- **Scientific Computing** — simulations, mathematical computations
- **Education** — learning programming fundamentals

Why Learn Python?

- **Readable syntax** — Python code looks almost like plain English
- **Beginner-friendly** — minimal setup, easy to learn
- **Versatile** — used in almost every field of technology
- **Large community** — millions of users, vast resources and libraries
- **Free and open source** — freely available for anyone to use

Installing Python

1. Visit https://www.python.org/downloads/
2. Download the latest version of Python 3 for your operating system
3. Run the installer — make sure to tick *"Add Python to PATH"*
4. Open a terminal (Command Prompt on Windows, Terminal on Mac/Linux) and type:

```
python --version
```

If you see something like `Python 3.11.0`, installation was successful.

Note

We will use **IDLE** (which comes with Python) or any text editor such as **VS Code** or **Thonny** (recommended for beginners) to write our programs.

Setting Up Your Python Environment

Before writing Python code, you need the right tools. The recommended setup for beginners — especially in scientific and data-related work — is **Anaconda**.

Anaconda Distribution

Anaconda is a free, open-source distribution of Python that comes bundled with:

- **Python** itself
- **conda** — a powerful package and environment manager
- **Hundreds of scientific packages** — NumPy, Pandas, Matplotlib, SciPy, and more
- **Anaconda Navigator** — a graphical interface to manage everything

You can download Anaconda from: https://www.anaconda.com/download/

What is `conda`?

`conda` is a **command-line package manager** that ships with Anaconda. It helps you:

- Install, update, and remove Python packages
- Create separate **environments** for different projects
- Manage dependencies so tools do not conflict with each other

Note

Python works with many outside packages (libraries). `conda` helps you manage all those tools in one place — keeping everything organised and compatible.

Anaconda Navigator

Anaconda Navigator is a **desktop visual user interface** that allows you to launch applications and easily manage conda packages, environments, and channels **without using command-line commands**.

Important

We use Navigator in this course! It provides a point-and-click way to work with packages and environments — no need to type `conda` commands in a terminal.

Why Use Navigator?

The Python team strongly recommends:

Official Recommendation

"If you are new to Python or the Scientific Python ecosystem, we strongly recommend you to install and use Anaconda. It comes with Spyder and all its dependencies, along with the most important Python scientific libraries (i.e. NumPy, Pandas, Matplotlib, IPython, etc) in a single, easy to use environment."

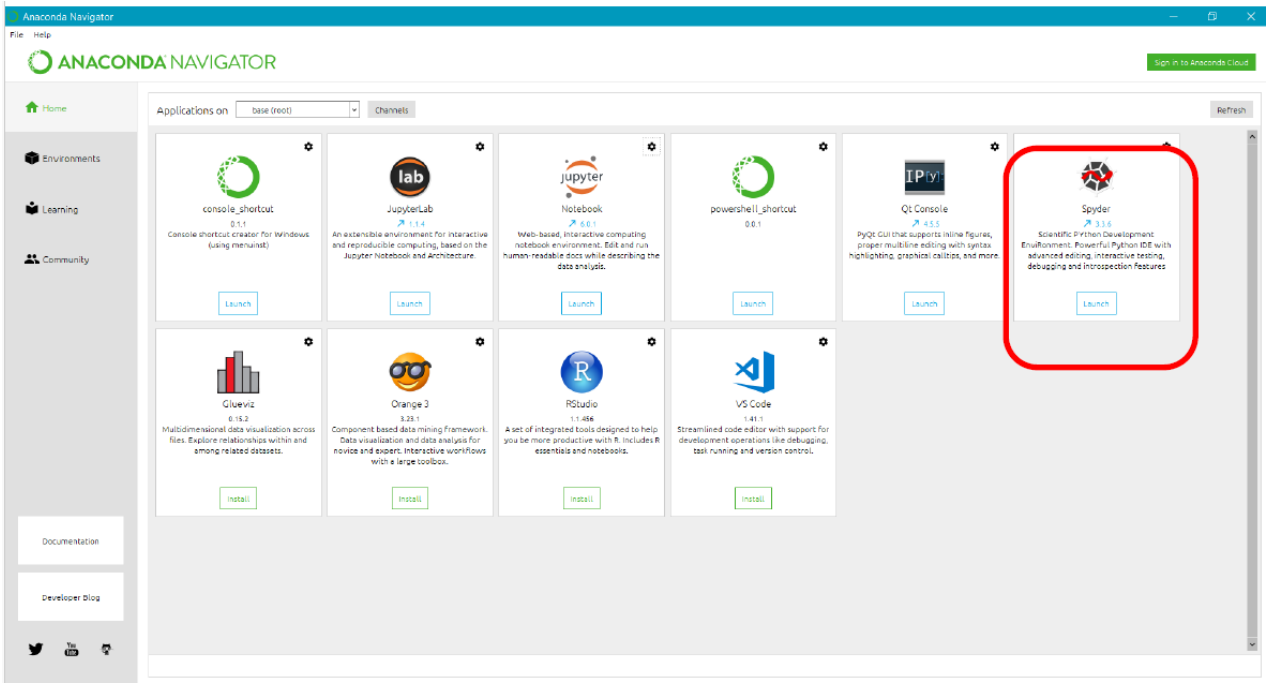
With Navigator you can:

- **Find** the packages you want
- **Install** them in an environment
- **Run** the packages directly
- **Update** them — all inside Navigator, no terminal needed

Some Notable Packages Available via Navigator

Package	Description
NumPy	Fast numerical computing with arrays and matrices
Pandas	Data manipulation and analysis
Matplotlib	Data visualisation and plotting
SciPy	Scientific and technical computing
TensorFlow	Machine learning and deep learning
Scikit-learn	Machine learning tools
Seaborn	Statistical data visualisation

Table 1: Commonly used Python packages available in Anaconda

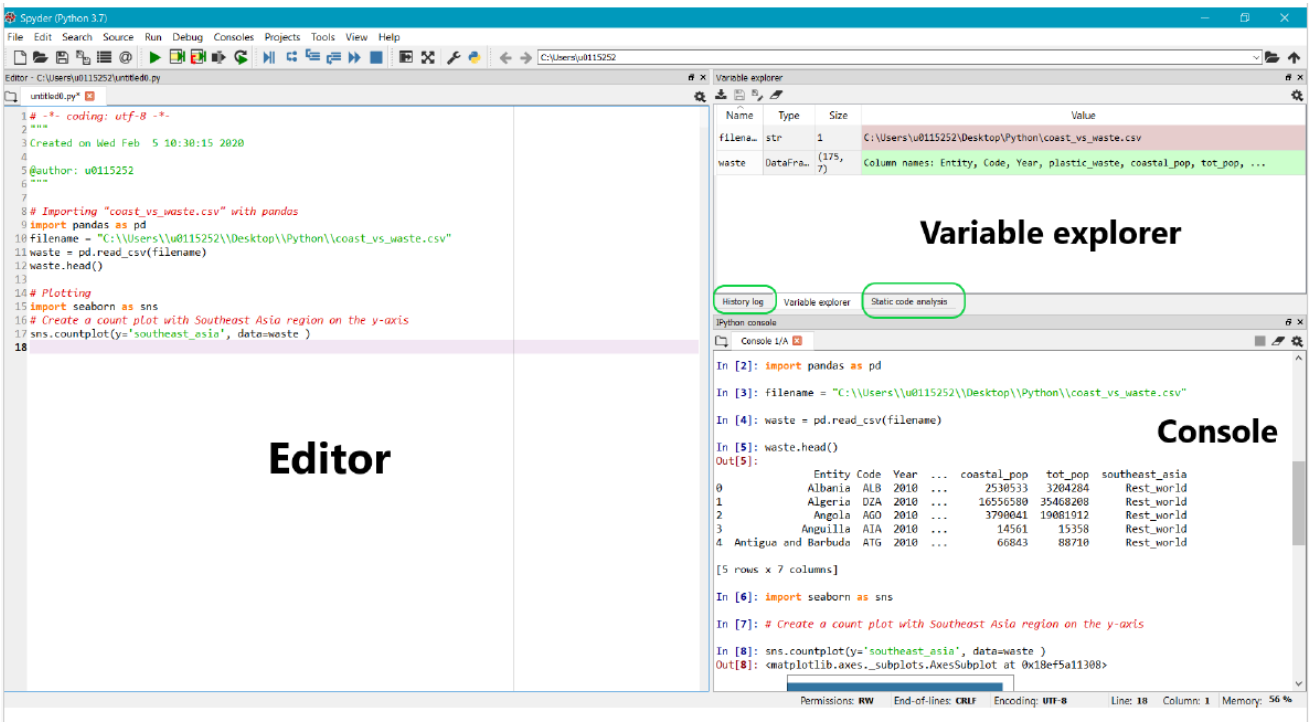


Spyder IDE

Spyder (Scientific Python Development Environment) is an **Integrated Development Environment (IDE)** for scientific computing. It is written in and designed for Python. You can launch it directly from Anaconda Navigator.

What is Spyder?

"Spyder is an Integrated Development Environment (IDE) for scientific computing, written in and for the Python programming language. It comes with an Editor to write code, a Console to evaluate it and view the results at any time, a Variable Explorer to examine the variables defined during evaluation, and several other facilities to help you effectively develop the programs you need as a scientist."



Spyder includes **multiple separate windows and tabs**. The three core components are:

1. Editor

The **Editor** is where you write your Python programs. It allows you to:

- Write sequences of commands that together make up a program
- Save your code as `.py` files
- Use syntax highlighting (colours that help you read code)
- Run your entire script or selected lines

2. Interactive Console (IPython Console)

The **Console** is where Python waits for you to type commands. It allows you to:

- Load data, perform calculations, and plot graphs interactively
- Test small pieces of code before adding them to your script
- See the output of each command immediately

3. Variable Explorer

The **Variable Explorer** lets you inspect any variables, functions, or objects created during your session. It shows:

- The **name** of each variable
- Its **type** (e.g. integer, string, DataFrame)
- Its **size** and **value**

4. History Log

The **History Log** stores the last 100 commands you have typed into the Console. This is useful for recalling and reusing previous commands.

Note

You can customise which windows are open and where they appear from:
`View` → `Toolbars` and `View` → `Window layouts`

Note

We may use Spyder for exercises, though it is not strictly required.

Jupyter Notebook

Jupyter Notebook is an open-source **web application** that allows you to create and share documents containing:

- **Live code** — code you can run directly in the browser
- **Equations** — mathematical notation
- **Visualisations** — graphs, charts, and plots
- **Narrative text** — written explanations using Markdown

Common uses of Jupyter Notebook include:

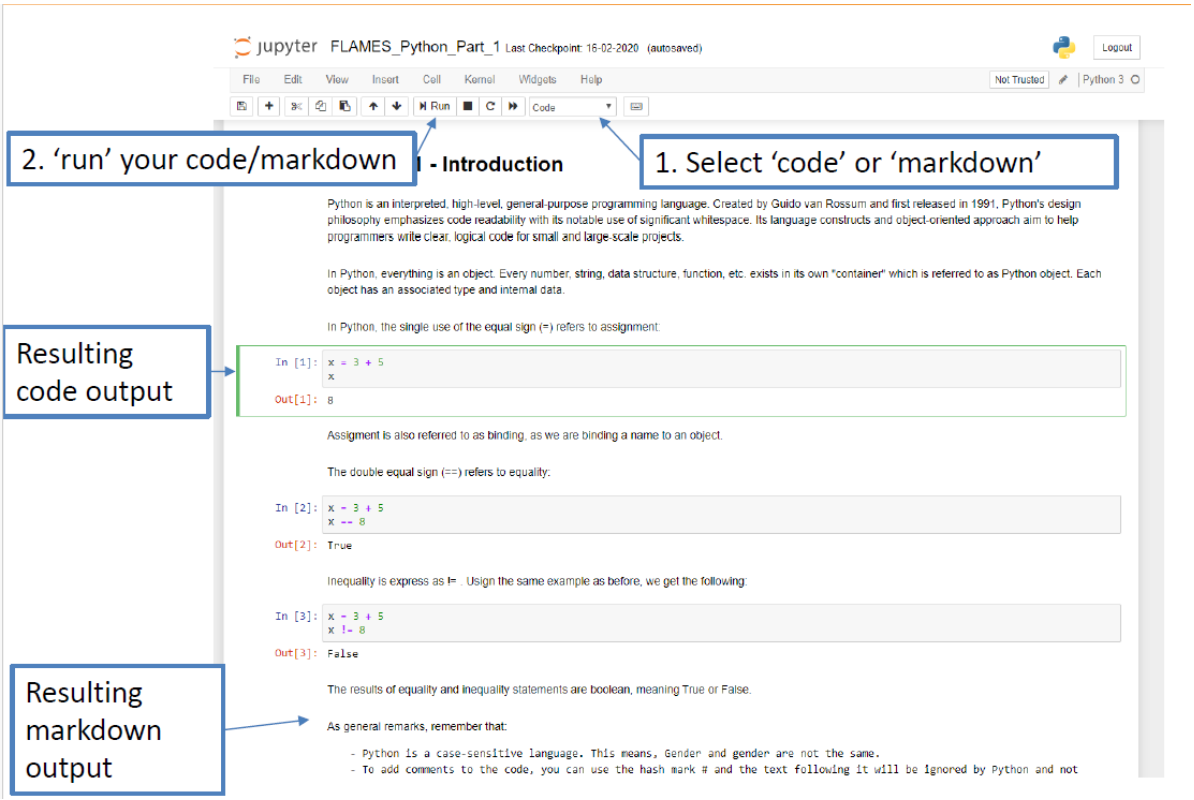
- Data cleaning and transformation
- Numerical simulation
- Statistical modelling
- Data visualisation
- Machine learning

Why Use Jupyter Notebook?

- **Shareable** — you can share your code with colleagues, along with results and explanations
- **Interactive** — the document is live; you can run, modify, and re-run code cells
- **Annotatable** — you can include results (e.g. graphs or tables) and add annotations written in **Markdown** language
- **Collaborative** — typically used among teams working on the same topic

For This Course

We will use **Jupyter Notebook** for the course material. You can add notes, experiment with extra code, and explore ideas as you like.



How Jupyter Notebook Works

A Jupyter Notebook is made up of **cells**. Each cell can be one of two types:
The basic workflow in Jupyter is:

1. Select the cell type — **Code** or **Markdown** — from the dropdown menu in the toolbar
2. Write your code or text in the cell
3. Click **Run** (or press `Shift + Enter`) to execute the cell
4. The **resulting output** (code result or rendered markdown) appears immediately below

Cell Type	Description
Code	Contains Python code. When you run it, the output appears directly below the cell.
Markdown	Contains formatted text, headings, bullet points, and equations. Used for notes and explanations.

Table 2: Jupyter Notebook cell types

```
x = 3 + 5
x
```

Listing 1: Example code cell in Jupyter

Output:

```
8
```

Note

In Jupyter Notebook, `In [1]:` labels the input cell and `Out[1]:` labels the output. The number in brackets indicates the order in which cells were run.

Comparing the Tools

	Spyder IDE	Jupyter Notebook
Best for	Writing and debugging scripts	Interactive exploration and sharing
Interface	Desktop application	Web browser
Output	Console pane	Inline below each cell
Sharing	Share `.py` files	Share `.ipynb` notebooks
Variable view	Built-in Variable Explorer	Use `print()` or extensions
Use in course	Exercises (optional)	Course material (primary)

Table 3: Spyder vs Jupyter Notebook

Your First Python Program

The traditional first program in any programming language is one that displays the message `Hello, Actuarial Science students!` on the screen. In Python, this is remarkably simple.

```
print("Hello,Actuarial Science students!")
```

Listing 2: Hello Actuarial Science students in Python

Output:

```
Hello, Actuarial Science students!
```

The `print()` function displays whatever is inside the parentheses on the screen.

```
print("Hello, Actuarial Science students!")
print("Welcome to Python.")
print("Let us begin!")
```

Listing 3: Printing multiple lines

Output:

```
Hello, Actuarial Science students!
Welcome to Python.
Let us begin!
```

Try It Yourself

Try printing your own name and the name of your school using the `print()` function.

Comments

A **comment** is a line in your code that Python ignores when running the program. Comments are used to explain what your code does. They are important for making your code readable and understandable.

In Python, a comment starts with the `#` symbol.

```
# This is a comment - Python will ignore this line
print("Hello")    # This prints Hello to the screen

# You can also use comments to disable code temporarily
# print("This line will not run")
```

Listing 4: Using comments

Note

Good programmers always comment their code. Get into the habit of explaining what each section of your code does.

Variables and Data Types

A **variable** is like a labelled box or container where you store data (information). You give the box a name and put something inside it.

Creating Variables

In Python, you create (declare) a variable simply by assigning a value to a name using the = sign:

```
name = "Alice"
age = 20
height = 1.75
is_student = True

print(name)
print(age)
print(height)
print(is_student)
```

Listing 5: Creating variables

Output:

```
Alice
20
1.75
True
```

Data Types

Every value in Python has a **data type**. The most common types are listed in Table 4.

Table 4: Common Python Data Types

Data Type	Description	Example
`int`	Whole numbers (integers)	`5, -3, 100`
`float`	Decimal numbers	`3.14, -0.5, 2.0`
`str`	Text (string of characters)	`"Hello", "Python"`
`bool`	True or False values	`True, False`

You can check the type of a variable using the `type()` function:

```
a = 10//3
b = 10 % 3
c = 10/3
x = 10
y = 3.14
z = "Hello"
w = True

print(type(a))    # <class 'int'>
print(type(b))    # <class 'int'>
print(type(c))    # <class 'float'>
```

```
print(type(x))    # <class 'int'>
print(type(y))    # <class 'float'>
print(type(z))    # <class 'str'>
print(type(w))    # <class 'bool'>
```

Listing 6: Checking data types

Variable Naming Rules

- Variable names can contain letters, numbers, and underscores (_)
- They **cannot** start with a number
- They **cannot** contain spaces (use _ instead, e.g., `first_name`)
- They are **case-sensitive**: `age` and `Age` are different variables
- They **cannot** be Python keywords (e.g., `if`, `for`, `while`)

Important

Avoid using Python reserved keywords as variable names. Examples of reserved keywords include: `if`, `else`, `for`, `while`, `True`, `False`, `None`, `and`, `or`, `not`, `return`, `print`.

Basic Input and Output

Output: `print()`

We have already seen the `print()` function. You can print multiple items by separating them with commas:

```
name = "Alice"
age = 20
print("Name:", name, "Age:", age)
```

Listing 7: Printing multiple values

Output:

```
Name: Alice Age: 20
```

You can also use **f-strings** (formatted strings) for cleaner output:

```
name = "Alice"
age = 20
print(f"My name is {name} and I am {age} years old.")
```

Listing 8: Using f-strings

Output:

```
My name is Alice and I am 20 years old.
```

Input: `input()`

The `input()` function allows you to get information from the user. Whatever the user types is stored as a **string**.

```
name = input("Enter your name: ")
print(f"Hello, {name}!")
```

Listing 9: Getting user input

Example Run:

```
Enter your name: Bob
Hello, Bob!
```

Note

`input()` always returns a **string**. If you want a number from the user, you must convert it using `int()` or `float()`:

```
age = int(input("Enter your age: "))
height = float(input("Enter your height in metres: "))
```

Arithmetic Operators

Python can perform mathematical calculations. The basic arithmetic operators are shown in Table 5.

Table 5: Python Arithmetic Operators

Operator	Description	Example	Result
`+`	Addition	`5 + 3`	8
`-`	Subtraction	`5 - 3`	2
`*`	Multiplication	`5 * 3`	15
`/`	Division	`5 / 2`	2.5
`//`	Floor Division	`5 // 2`	2
`%`	Modulus (remainder)	`5 % 2`	1
`**`	Exponentiation	`2 ** 3`	8

```
a = 10
b = 3

print(a + b)     # 13
print(a - b)     # 7
print(a * b)     # 30
print(a / b)     # 3.3333...
```

```
print(a // b)    # 3  (drops the decimal)
print(a % b)     # 1  (remainder of 10 / 3)
print(a ** b)    # 1000 (10 to the power of 3)
```

Listing 10: Arithmetic operations

Try It Yourself

Write a Python program that asks the user to enter two numbers, then prints their sum, difference, product, and quotient.

Data Structures

In this chapter, we will focus on Python data structures: lists, tuples and dicts. Data structures in Python are simple but powerful.

Lists

A **list** is an ordered collection of items. Lists can hold any type of data, and items are separated by commas inside square brackets `[ ]`.

```
fruits = ["apple", "banana", "mango", "orange", "grape"]

print(fruits[0])     # apple   (first item, index 0)
print(fruits[2])     # mango   (third item, index 2)
print(fruits[-1])    # grape   (last item)
print(len(fruits))   # 5       (number of items)
```

Listing 11: Creating and accessing a list

Modifying Lists

```
numbers = [3, 1, 4, 1, 5, 9, 2]

numbers.append(6)        # add 6 to the end
numbers.remove(1)        # remove first occurrence of 1
numbers.sort()           # sort in ascending order

print(numbers)
```

Listing 12: Common list operations

Output:

```
[1, 2, 3, 4, 5, 6, 9]
```

Tuple

Tuples are essentially lists. Tuples are used to group objects of different types. In contrast to lists, tuples cannot be modified after creating them (i.e. they are immutable).

We can rewrite our first list as a tuple by using round brackets or by invoking tuple.

```
first_tuple = (2, 4, 6, 8, 10)
first_tuple
type(first_tuple)
tup = tuple(['hello', 2, 4, True])
tup
tup[3] = "False" # If we try to change the fourth element of
   tuple, we get an error message
```

Listing 13: Tuple

Indeed, once the tuple is created, it is not possible to modify the object is stored in each slot.

There are only two methods for tuples of numbers, since the size and content of a tuple cannot be modified. The first method is count, which counts how many times a specific value is present in the tuple.

```
num_tup = (2, 4, 6, 8, 8, 8, 10)
num_tup.count(8)
```

The second method is index, which tells us at what index the given number first appears in the tuple.

```
num_tup.index(8)
```

Dictionary

Dictionaries or dicts are very important Python data structures. Dictionaries are collections of content in the form of key-value pairs.

You can create dictionaries by using the command dict, or by using curly brackets { } in association with the column operator ':' to separate keys and values.

```
dict_course = dict(organizer = "Flames", topic = "programming",
   software = "Python", level = "introduction")
dict_course
```

```
course = {'organizer':'Flames', 'topic':'programming', 'software'
   :'Python', 'level':'introduction'}
course
```

To access or insert elements inside a dictionary, use square brackets.

```
course['topic']
```

You can easily add a key-value pair to an existing dictionary.

```
course['participants'] = 45
course
```

Use the del keyword or the pop method to delete a key-value pair.

```
del course['participants']
course
```

Or

```
course.pop('organizer')
course
```

You can extract all keys from the dictionnary using the keys method.

```
course.keys()
type(course.keys())
```

The result is stored in an object of type dict_keys.

You can convert it to a list using the list function, if needed.

```
list_course = list(course)
list_course
type(list_course)
```

Important remark: keys of a dict have to be immutable objects (int, float, str) or tuples. You can check whether an object can be used as a key in a dict with the hash function:

```
hash([2, 4, 6]) # error because lists are mutable!
```

Example 2

```
scores = {" Maths ": 78, " Economics ": 85, " Stats ": 90,
" Python ": 88, " English ": 72}

dict_scores = dict(Maths = 78, Economics = 85, Stats = 90, Python
    = 88, English = 72)
dict_scores

print ( list ( scores . keys ())) # All keys

average = sum ( scores . values ()) / len ( scores ) # Average
print (f" Average : { average :.1f}")

best = max (scores , key = scores .get ) # highest score
print (f" Highest : { best } ({ scores [ best ]})")
```

Conditional and Iterative Processing

Conditional statements allow your program to make **decisions** — to execute different blocks of code depending on whether a condition is true or false.

Python has several built-in keywords for conditional logic and loops.

Conditional Processing: `if`, `elif`, and `else` Statement

The `if` Statement

The if statement is one of the most important control flow statement types. We use the if statement combined with a condition/statement that returns True of False. If that condition is True, then the code in the block that follows is evaluated. If the condition is False, then the code is not evaluated.

```
age = 18

if age >= 18:
    print("You are an adult.")
```

Output:

```
You are an adult.
```

```
x = -5
if x < 0:
    print("negative number")
```

Output:

```
negative number.
```

The `if-else` Statement

Optionally, you can provide code that has to be evaluated when the condition is false using else.

```
age = 15

if age >= 18:
    print("You are an adult.")
else:
    print("You are a minor.")
```

Output:

```
You are a minor.
```

```
x = 1
if x < 0:
    print("negative number")
else:
    print("nonnegative number")
```

Output:

```
nonnegative number
```

The `if-elif-else` Statement

Use `elif` (short for "else if") when you have **more than two** possible outcomes: You can use one or more `elif` blocks, but the final statement has to be `else`. The conditions are evaluated in order. If one of them is True, the subsequent code is evaluated and no further `elif` and else blocks will be evaluated.

```
x = 160
if x > 100:
    print("this number is big")
elif x > 10:
    print("this number is not big and not small")
else:
    print("this number is small")
```

Output:

```
this number is big
```

```
score = 75

if score >= 80:
    print("Grade: A")
elif score >= 70:
    print("Grade: B")
elif score >= 60:
    print("Grade: C")
elif score >= 50:
    print("Grade: D")
else:
    print("Grade: F")
```

Output:

```
Grade: B
```

Comparison Operators

Table 6: Comparison Operators

Operator	Meaning	Example
`==`	Equal to	`5 == 5` is `True`
`!=`	Not equal to	`5 != 3` is `True`
`>`	Greater than	`7 > 3` is `True`
`<`	Less than	`3 < 7` is `True`
`>=`	Greater than or equal	`5 >= 5` is `True`
`<=`	Less than or equal	`4 <= 5` is `True`

Important

Do not confuse = (assignment) with == (comparison). Writing `if x = 5` is a syntax error. The correct form is `if x == 5`.

Iterative Processing: for and while

In chapter 3, we saw how to create, access, and manipulate lists, tuples, and dictionaries. In case we want to perform more than one operation on a collection of data contained in a list for example, we can use loops. First we create some patient data (patient IDs, type of cronic disease, cholesterol values):

```
patient = []
patient1 = ["ID0101", "diabetes", 239]
patient.append(patient1)
patient2 = ["ID0102", "cancer", 201]
patient.append(patient2)
patient3 = ["ID0103", "epilepsy", 171]
patient.append(patient3)
patient4 = ["ID0104", "allergy", 163]
patient.append(patient4)
patient5 = ["ID0105", "hypertension", 252]
patient.append(patient5)
patient
```

```
type(patient)
```

Loops

A **loop** allows you to repeat a block of code multiple times without writing it over and over again.

The `for` Loop

A `for` loop repeats a block of code for each item in a sequence.
Why use loops? If you want to avoid to write the same code over and over again, you might want to use a `for` loop. Below you can find one of the simplest examples of using a for loop.

```
cities = ["New York", "London", "Paris", "Brussels", "Tokyo"]
for c in cities:
    print(c)
```

Output:

```
New York
London
Paris
Brussels
Tokyo
```

For example, if we want to access the average cholesterol values of all patients in the list that we've just created, we can use the following code:

```
tot_chol = 0.0
for p in patient:
    chol = p[2]
    tot_chol += chol
tot_chol

avg_chol = tot_chol / len(patient)
avg_chol
```

Output:

```
205.2
```

The average cholosterol level in our patient data is 205.2. Thanks to using a for loop, we saved time in computing the mean. However, what we really have done here?

First we create the variable that will contain the sum of all cholesterol levels and we initialize it to zero.

Then we use a for loop. The syntax says that for every p in patient, the cholesterol values (in position 2) are accessed and then the total chol level will be incremented by the individual cholesterol levels within the loop.

Note

Two remarks about the code:

- Using the letter p is arbitrary. You could use any other letter or word. However, using the first letter of the list you are looping is the convention.
- The operator $+=$ is the addition assignment operator in Python. It basically adds the value of the second variable to the first one, and assigns the result to the first variable: $A+=B$ is the same as to $A = A + B$.

```
for i in range(5):
    print(i)
```

Output:

```
0
1
2
3
4
```

Note

`range(5)` generates the numbers `0, 1, 2, 3, 4` (it starts at 0 and goes up to, but **not including**, 5).

- `range(n)` $\rightarrow$ 0 to $n-1$

- `range(a, b)` → a to $b-1$
- `range(a, b, step)` → a to $b-1$ in steps of `step`

```
fruits = ["apple", "banana", "mango", "orange"]

for fruit in fruits:
    print(fruit)
```

Output:

```
apple
banana
mango
orange
```

The `while` Loop

A `while` loop repeats a block of code **as long as** a condition is true.

It is like a repeated `if` statement. It specifies a condition and a block of code that is executed over and over again as long as the condition is True.

```
count = 1

while count <= 5:
    print(f"Count is {count}")
    count = count + 1

print("Done!")
```

Output:

```
Count is 1
Count is 2
Count is 3
Count is 4
Count is 5
Done!
```

```
count = 5
while count < 9:
    print ('The count is:', count)
    count = count + 1

print ("Good bye!")
```

Output:

```
The count is 5
The count is 6
The count is 7
```

```
The count is 8
Good bye!
```

Important

Be careful with `while` loops! If the condition never becomes `False`, the loop runs forever (an **infinite loop**). Always make sure the loop has a way to end.

Functions

A **function** is a reusable block of code that performs a specific task. Instead of writing the same code multiple times, you define it once and call it whenever you need it.

Defining and Calling a Function

```python
def greet():
    print("Hello! Welcome to Python.")

# Call the function
greet()
greet()
```

Listing 14: Defining and calling a function

Output:

```
Hello! Welcome to Python.
Hello! Welcome to Python.
```

Functions with Parameters

You can pass information into a function using **parameters**:

```python
def greet(name):
    print(f"Hello, {name}! Welcome to Python.")

greet("Alice")
greet("Bob")
```

Listing 15: Function with parameters

Output:

```
Hello, Alice! Welcome to Python.
Hello, Bob! Welcome to Python.
```

Functions that Return a Value

A function can also **return** a result using the `return` keyword:

```python
def add(a, b):
    result = a + b
    return result

sum1 = add(3, 5)
sum2 = add(10, 20)

print(sum1)    # 8
print(sum2)    # 30
```

Listing 16: Function with return value

```python
import math

def circle_area(radius):
    area = math.pi * radius ** 2
    return area

r = 7
print(f"Area of circle with radius {r} = {circle_area(r):.2f}")
```

Listing 17: Function to compute area of a circle

Output:

```
Area of circle with radius 7 = 153.94
```

Try It Yourself

Write a function called `is_even(n)` that takes an integer n and returns `True` if n is even and `False` otherwise. Test it with several values.

Looping Through a List

```python
marks = [85, 72, 90, 68, 77]
total = 0

for mark in marks:
    total += mark

average = total / len(marks)
print(f"Average mark: {average}")
```

Listing 18: Iterating over a list

Output:

```
Average mark: 78.4
```

Putting It All Together: A Complete Example

The following program uses variables, input, a loop, conditionals, and functions all together:

```
def get_grade(score):
    if score >= 80:
        return "A"
    elif score >= 70:
        return "B"
    elif score >= 60:
        return "C"
    elif score >= 50:
        return "D"
    else:
        return "F"

# Get number of students
n = int(input("How many students? "))
scores = []

for i in range(n):
    score = float(input(f"Enter score for student {i+1}: "))
    scores.append(score)

# Display results
print("\n--- Results ---")
for i, score in enumerate(scores):
    grade = get_grade(score)
    print(f"Student {i+1}: Score = {score:.1f}, Grade = {grade}")

# Compute average
average = sum(scores) / len(scores)
print(f"\nClass average: {average:.2f}")
```

Listing 19: Student grade calculator

Reading & Writing Data Files

When working with data, the first step is often to load them into Python. First, the file location is specified, then a stream to that location is created.

Importing text files

Python's basic open function is what you need. It is used to open a connection to a file. First assign the filename to a variable as a string.
Important remark: use a double backslash in file paths on Windows!

```
filename = "C:\\Users...\\chol.txt" # use this on Windows
```

Then pass filename to the open function together with the argument *mode = 'r'*, which makes sure that we can only read the file and not write to it. If you want to open a file to write to it, you should pass the argument *mode='w'*.

```
file = open(filename, mode = 'r')
```

Next assign the text from the file to a variable text by applying the method read to the connection to the file.

```
text = file.read()
text
```

Always make sure you have closed the connection to the file using the close method.

```
file.close()
```

Importing flat files

Flat files are basic text files containing records. We can use `pandas` to import flat files if we want to store the data in a DataFrame

Importing flat files with pandas

Why do you need to import files with pandas? Because the majority of data sets are multidimensional, where columns might be of different types.

The data structure most relevant to data manipulation and analysis that pandas offers, is the DataFrame (analogous to dataframe in R).

A data frame has observations (rows) and variables (columns), which is exactly what we need to perform statistical analysis.

To use pandas you first have to import it. If you wish to import a .csv file, you only need to call the function read_csv and supply it with the name of the file.

```
import pandas as pd     # first import pandas
filename = "C:\\Users...\\rainfall.csv"     # pass the path to
   your file
data = pd.read_csv(filename)     # assign the dataframe to
   variable data
data.head()     # check the first five rows of the DataFrame
```

```
type(data)
```

```
data = pd.read_csv(filename, index_col = 0) # tells Pandas to use
    the first column (column number 0) as the row labels (index)
data.tail(15)  # displays the last 15 rows
```

Writing files

If we want to export new data, e.g. the pandas DataFrame object data that we created in the previous section, we can use the following methods.

```
type(data)
```

```
# if you do not specify a path, the file will be saved in the
   folder of your jupyter notebook
out_csv = 'data.csv'
data.to_csv(out_csv)

out_xlsx = 'data.xlsx'
data.to_excel(out_xlsx)

# here is an example where you specify the whole path
data.to_excel("C:\\Users...\\data1.xlsx")
```

NumPy

In general, a Python library is a collection of functions and methods that allows you to perform actions without writing your own code. If you are working with data, some libraries are foundational: `NumPy, pandas, Matplotlib,` ...

`NumPy` , short for Numerical Python, is one of the most important foundational packages for numerical computing in Python. The aim of this chapter is to give an understanding of `NumPy` arrays and array-oriented computation, which will help you use tools with array-oriented semantics, like `pandas` , in a more effective way.

`NumPy` provides an alternative to the Python list data structure: `NumPy` array. It is similar to a list but it has additional features, such as the ability to perform calculation over entire arrays in an easy and fast way. It is not part of the core Python installation (download from www.python.org), so you have to download it. However, `NumPy` is part of the "Anaconda distribution" (download from).

You can import the `NumPy` package to use it in your Python session; np is the standard convention used to import `NumPy` . You can decide to not follow this convention, but we will in this manual.

```
import numpy as np
```

NumPy arrays

We will cover only a small portion of NumPy features in this chapter.
The first important concept we introduce is an array. To create a `NumPy` array, you can use the array function and simply specify a regular Python list as input data.

```
height_array = np.array([1.55, 1.78, 1.65, 1.67, 1.88, 1.71])  #
    height of people
height_array
```

Output:

```
array([1.55, 1.78, 1.65, 1.67, 1.88, 1.71])
```

```
weight_array = np.array([52.1, 66.3, 65.8, 55.9, 84.5, 67.3])  #
    weight of people
weight_array
```

Output:

```
array([52.1, 66.3, 65.8, 55.9, 84.5, 67.3])
```

If we want to calculate the Body Mass Index (BMI) of this group of people, we can do it as follows:

```
bmi = weight_array / height_array ** 2
bmi
```

Output:

```
array([21.68574402, 20.92538821, 24.16896235, 20.04374485,
    23.90787687, 23.01562874])
```

The calculations with arrays were performed element-wise.

If you would have tried to do the same calculation with regular lists, you would get an error!

```
# bmi calculation with lists
height = [1.55, 1.78, 1.65, 1.67, 1.88, 1.71]
weight = [52.1, 66.3, 65.8, 55.9, 84.5, 67.3]
weight / height ** 2 # ERROR!
```

Recall that the `+` operator for lists concatenates the lists. For arrays, however, it computes the element-wise addition.

```
height + weight # concatenation
```

Output:

```
[1.55, 1.78, 1.65, 1.67, 1.88, 1.71, 52.1, 66.3, 65.8, 55.9,
    84.5, 67.3]
```

```
height_array + weight_array # element-wise addition
```

Output:

```
array([53.65, 68.08, 67.45, 57.57, 86.38, 69.01])
```

Besides the fact that `NumPy` operations tend to act element-wise, there is one other important consideration to keep in mind:

NumPy assumes that your NumPy array only contains values of a single type. If you try to create an array with different types, the resulting NumPy array will contain a single type. In the following example, the Boolean and the float are converted to strings:

```
np.array([7.5, "today", False])
```

Output:

```
array(['7.5', 'today', 'False'], dtype='<U32')
```

In general, you can work with NumPy arrays in the same as you can with regular Python lists.

```
# print the array
height_array
```

Output:

```
array([1.55, 1.78, 1.65, 1.67, 1.88, 1.71])
```

```
# access the first element
height_array[0]
```

Output:

```
1.15
```

There is specific way to subset a NumPy array using an array of Booleans. This does not work for lists.

```
height_array < 1.7
```

Output:

```
array([ True, False,  True,  True, False, False])
```

```
height_array[height_array <= 1.67]   #Select only the elements
   whose condition is True
```

Output:

```
array([1.55, 1.65, 1.67])
```

```
height < 1.7 ##ERROR
```

2D NumPy arrays

Take a look at the type of NumPy array height that we created in the previous section.

```
type(height_array)
```

Output:

```
numpy.ndarray
```

The output indicates it is a type defined in the `NumPy` package, where `ndarray` stands for "n-dimensional array".
Our array height_array is a one-dimesional array, but is it possible to create two, three, ..., or n-dimensional arrays! You can create 2D arrays from a regular Python list of lists.

```
array_2d = np.array([[1.55, 1.78, 1.65, 1.67, 1.88, 1.71],
                     [52.1, 66.3, 65.8, 55.9, 84.5, 67.3]])
array_2d
```

Output:

```
array([[ 1.55,  1.78,  1.65,  1.67,  1.88,  1.71],
    [52.1 , 66.3 , 65.8 , 55.9 , 84.5 , 67.3]])
```

We obtain a data structure where each sublist in the list corresponds to a row in the 2-dimensional array. You can obtain the number of rows and columns using the shape attribute.

```
array_2d.shape
```

Output:

```
(2, 6)
```

```
array_2d[1] # select the second row
```

Output:

```
array([52.1, 66.3, 65.8, 55.9, 84.5, 67.3])
```

```
print(array_2d[1][3])#select the fourth element in the second row
print(array_2d[0, 2]) # select the third element in the first row
```

Output:

```
55.9
1.65
```

```
array_2d[:, 1:3] # to select second & third elements of both rows
```

Output:

```
array([[ 1.78,  1.65],
       [66.3 , 65.8 ]])
```

Basic Statistics with NumPy

You can generate useful summary statistics about you data with NumPy.

```
Mean=np.mean(height_array) # mean
print(Mean)
Median=np.median(height_array) # median
print(Median)
Variance=np.var(height_array) # variance
print(Variance)
```

```
Std=np.std(height_array) # standard deviation
print(Std)
Min=np.min(height_array) # minimum
print(Min)
Max=np.max(height_array) # maximum
print(Max)
Cor=np.corrcoef(height_array, weight_array) # correlation
print(Cor)
```

Output:

```
1.706666666666667
1.69
0.010755555555555551
0.10370899457402695
1.55
1.88
[[1.         0.90378738]
 [0.90378738 1.        ]]
```

We can even compute the mean/median/var/std/min/max of all rows or columns in a 2d array using the axis option.

```
Mean_Wgt_Hgt=np.mean(array_2d) # Computes the overall mean of all
    values in the entire array (does not make much sense)
print(Mean_Wgt_Hgt)
Mean_Wgt_Hgt_pp=np.mean(array_2d, axis = 0) # mean of weight and
   height per person- Computes the mean column by column(does not
   make much sense either)
print(Mean_Wgt_Hgt_pp)
Mean_Hgt_Wgt=np.mean(array_2d, axis = 1) # mean height and mean
   weight -Computes the mean row by row
print(Mean_Hgt_Wgt)
```

Output:

```
33.51166666666667
[26.825 34.04  33.725 28.785 43.19  34.505]
[ 1.70666667 65.31666667]
```

pandas

Python has been great for data munging and preparation, but less so for data analysis and modeling. Pandas helps fill this gap, enabling you to carry out your entire data analysis workflow in Python without having to switch to a more domain specific language like R.

The import convention for `pandas` is to import it as `pd`. So every time you encounter pd in the code, it is referring to `pandas`.

```
import pandas as pd
```

pandas Data Structures

In this section we will focus on two important data structures in `pandas`: **Series** and **DataFrame**.

Series

A `Series` is a one-dimensional array-like object containing a sequence of values and an associated array of data labels, called its index.

Another way to think about a `Series` is a fixed-length, ordered *dict*, as it is a mapping of index values to data values. It can be used in those situations where you can use a *dict*.

You can create a `Series` from data contained in a *dict*.

```
prison_data = {'Belgium':11.071, 'France':71.190, 'Italy':54.653,
    'Spain':61.526}

first_series = pd.Series(prison_data)
first_series
```

Output:

```
Belgium     11.071
France      71.190
Italy       54.653
Spain       61.526
dtype: float64
```

DataFrame

A `DataFrame` contains an ordered collection of columns, each of which can be a different value type (numeric, string, boolean, ...). It has both a row and column index. Simply, if you paste together several Series, you obtain a `DataFrame`.

There are many ways to construct a `DataFrame`, such as starting from a `dict`.

```
country_dict = {
'country':['Belgium', 'France', 'Italy', 'Spain'],
'area':[30.689, 643.801, 301.338, 505.990],
'prison_pop':[11.071, 71.190, 54.653, 61.526],
'population':[11.31, 66.60, 60.67, 44.44]
}

first_df = pd.DataFrame(country_dict)
first_df
```

Output:

```
        country    area      prison_pop    population
0       Belgium    30.689    11.071        11.31
```

```
1       France     643.801   71.190        66.60
2       Italy      301.338   54.653        60.67
3       Spain      505.990   61.526        44.44
```

The data structure is clear: the rows represent the observations and the columns represent the variables.

Each row has an automatically assigned label (from 0 to 3).

If you want to set them manually, use the following syntax:

```
first_df.index = ["BE", "FR", "IT", "SP"]
first_df
```

Output:

```
        country    area       prison_pop   population
BE      Belgium    30.689     11.071       11.31
FR      France     643.801    71.190       66.60
IT      Italy      301.338    54.653       60.67
SP      Spain      505.990    61.526       44.44
```

Of course, you can import your dataset in Python by using several `pandas` import functions, which we saw how to do in lecture 7.

Here we want to import a new dataset, which comes from the USDA (United States Department of Agriculture) and it contains information about milk production in the USA.

```
import pandas as pd
filename = "C:\\Users\\...\\state_milk_production.csv"
milk = pd.read_csv(filename)
```

```
milk.tail(5)
milk.head(5)
```

Output:

```
        region          state           year     milk_produced
0       Northeast       Maine           1970.0   6.190000e+08
1       Northeast       New Hampshire   1970.0   3.560000e+08
2       Northeast       Vermont         1970.0   1.970000e+09
3       Northeast       Massachusetts   1970.0   6.580000e+08
4       Northeast       Rhode Island    1970.0   7.500000e+07
```

You can use the info method to obtain information on a `DataFrame`.

```
milk.info()
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2400 entries, 0 to 2399
Data columns (total 4 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   region          2400 non-null   object
 1   state           2400 non-null   object
 2   year            2400 non-null   float64
 3   milk_produced   2400 non-null   float64
dtypes: float64(2), object(2)
memory usage: 75.1+ KB
```

You can use *dtypes* to see the data types for each variable

```
milk.dtypes
```

Output:

```
region            object
state             object
year             float64
milk_produced    float64
dtype: object
```

Get a list of all variables

```
milk.columns
```

Output:

```
Index(['region', 'state', 'year', 'milk_produced'], dtype='object
    ')
```

There are multiple ways to index and select data from a `DataFrame`: using [ ]

To access only the "state" column, we can use its actual name, as shown here:

```
milk["state"].head(10)
```

Output:

```
0            Maine
1    New Hampshire
2          Vermont
3    Massachusetts
4     Rhode Island
5      Connecticut
6         New York
7       New Jersey
8     Pennsylvania
9         Delaware
Name: state, dtype: object
```

Note that the last line in the output tells us that the *dtype* is object. Which type of object is returned here? We can find out with the type function.

```
type(milk["state"])
```

Output:

```
pandas.core.series.Series
```

It turns out to be a pandas `Series`, i.e. a 1D labelled array (see previous section).

A second way to extract one variable as a pandas `Series` is the following:

```
milk.state
```

Output:

```
0                Maine
1        New Hampshire
2              Vermont
3        Massachusetts
4         Rhode Island
             ...
2395        Washington
2396            Oregon
2397        California
2398            Alaska
2399            Hawaii
Name: state, Length: 2400, dtype: object
```

```
type(milk.state)
```

Output:

```
pandas.core.series.Series
```

To select the "state" column and to keep it in a `DataFrame` format, you need double squared brackets.

```
milk[["state"]].head()
```

Output:

```
         state
0        Maine
1        New Hampshire
2        Vermont
3        Massachusetts
4        Rhode Island
```

```
type(milk[["state"]])
```

Output:

```
pandas.core.series.Series
```

You can also access rows by using [ ]. If we want to access the rows between the second and the fifth, we use the following syntax:

```
milk[1:6]
```

Output:

	region	state	year	milk_produced
1	Northeast	New Hampshire	1970.0	3.560000e+08
2	Northeast	Vermont	1970.0	1.970000e+09
3	Northeast	Massachusetts	1970.0	6.580000e+08
4	Northeast	Rhode Island	1970.0	7.500000e+07
5	Northeast	Connecticut	1970.0	6.610000e+08

Manipulating `DataFrames` with pandas

Filtering `DataFrames`

Filtering is useful when you wish to select part of the data based on specific properties.

Behind filtering there is the idea of a Boolean Series.

```
milk.head()
```

```
new_milk = milk[milk.milk_produced > 4.0000000e+10]
new_milk.head()
```

Output:

	region	state	year	milk_produced
1897	Pacific	California	2007.0	4.068300e+10
1947	Pacific	California	2008.0	4.120300e+10
2047	Pacific	California	2010.0	4.038500e+10
2097	Pacific	California	2011.0	4.146200e+10
2147	Pacific	California	2012.0	4.180100e+10

```
milk[milk.year == 2010].head()
```

Output:

	region	state	year	milk_produced
2000	Northeast	Maine	2010.0	5.870000e+08
2001	Northeast	New Hampshire	2010.0	2.940000e+08
2002	Northeast	Vermont	2010.0	2.521000e+09
2003	Northeast	Massachusetts	2010.0	2.420000e+08
2004	Northeast	Rhode Island	2010.0	2.000000e+07

Filters can be combined using the standard logical operators AND (& in Python) and OR (| in Python).

```
# using AND (&): both conditions have to be True
milk[(milk.milk_produced > 2.0000000e+10) & (milk.year == 2015)]
```

Output:

```
        region             state          year    milk_produced
2262    Lake States        Wisconsin      2015.0  2.903000e+10
2297      Pacific          California     2015.0  4.089700e+10
```

```
# using OR (|): at least one of the conditions has to be True
milk[(milk.milk_produced > 2.0000000e+10) | (milk.year == 2015)].
   tail(10)
```

Output:

```
        region        state       year    milk_produced
2294    Mountain      Nevada      2015.0  6.690000e+08
2295    Pacific       Washington  2015.0  6.606000e+09
2296    Pacific       Oregon      2015.0  2.551000e+09
2297    Pacific       California  2015.0  4.089700e+10
2298    Pacific       Alaska      2015.0  4.000000e+06
2299    Pacific       Hawaii      2015.0  3.500000e+07
2312    Lake States   Wisconsin   2016.0  3.011000e+10
2347    Pacific       California  2016.0  4.046900e+10
2362    Lake States   Wisconsin   2017.0  3.032000e+10
2397    Pacific       California  2017.0  3.979800e+10
```

You can filter a column based on another one. For example, you can select the years in which the quantity of milk produced is larger than $3.0000000e + 10$.

```
milk.year[milk.milk_produced > 3.0000000e+10]
```

Dealing with `NaNs`

Missing values are common in datasets. In Python, missing values are represented by `NaN`, which stands for 'not a number'.

To show you how to deal with `NaN`, we add a new column about life expectancy that contains two missing values to our country dictionary. We need `NumPy` to add missing values.

```
import numpy as np
country_df = pd.DataFrame({
'country':['Belgium', 'France', 'Italy', 'Spain'],
'area':[30.689, 643.801, 301.338, 505.990],
'prison_pop':[11.071, 71.190, 54.653, 61.526],
'population':[11.31, 66.60, 60.67, 44.44],
'life_exp': [80.99, np.nan, 82.54, np.nan]
})

country_df
```

Output:

```
         country  area      prison_pop   population      life_exp
0        Belgium   30.689         11.071      11.31         80.99
1        France   643.801         71.190      66.60         NaN
2        Italy    301.338         54.653      60.67         82.54
3        Spain    505.990         61.526      44.44         NaN
```

Given a `DataFrame`, how can we figure out which rows or which columns contain missing values?

We can use the *isnull* method to indicate where entries are missing.

```
country_df.isnull()
```

Output:

```
         country  area prison_pop population     life_exp
0        False    False     False      False        False
1        False    False     False      False        True
2        False    False     False      False        False
3        False    False     False      False        True
```

The *any* method acts on columns or rows and returns True if and only if at least one of the entries is True.

It acts on columns (axis = 0) by default, but acts on rows if you specify axis = 1.

```
country_df.isnull().any()
```

Output:

```
country        False
area           False
prison_pop     False
population     False
life_exp        True
dtype: bool
```

```
country_df.isnull().any(axis = 0)
```

Output:

```
country        False
area           False
prison_pop     False
population     False
life_exp        True
dtype: bool
```

```
country_df.isnull().any(axis = 1)
```

Output:

```
0    False
1     True
2    False
3     True
dtype: bool
```

If you are interested in the number of missing entries in each column, you can use the combination `isnull().sum()` and apply it to the entire `DataFrame`.

```
country_df.isnull().sum()
```

Output:

```
country        0
area           0
prison_pop     0
population     0
life_exp       2
dtype: int64
```

The methods *dropna* removes all rows with missing values. Proceeding with this data set leads to a **complete case analysis**.

```
country_df.dropna()
```

Output:

```
                country area    prison_pop      population
                   life_exp
0        Belgium 30.689  11.071  11.31   80.99
2        Italy   301.338 54.653  60.67   82.54
```

Transforming `DataFrames`

We can transform our data using elementary arithmetic operations. For example, suppose we want to divide the prison population by 2 in our country `DataFrame`.

We can use the pandas `DataFrame` vectorized method for floor division called *floordiv*. This arithmetic operation is applied to every entry in the `DataFrame` without using a loop.

```
country_df['prison_pop_divided'] = country_df.prison_pop.floordiv
   (2)  # save the values into a new variable
country_df
```

Output:

```
       country   area   prison_pop   pop  life_exp prison_pop_div
0       Belgium   30.689  11.071   11.31   80.99          5.0
1       France   643.801  71.190   66.60     NaN          35.0
2       Italy    301.338  54.653   60.67   82.54          27.0
3       Spain    505.990  61.526   44.44     NaN          30.0
```

As an alternative, you can use the `NumPy` vectorized function *np.floor_divide*.

```
import numpy as np
out = np.floor_divide(country_df.prison_pop, 2)  # NOT saving the
    results into a new variable
out
```

Output:

```
0     5.0
1    35.0
2    27.0
3    30.0
Name: prison_pop, dtype: float64
```

Statistics with pandas

We can use pandas statistical functions to obtain summary statistics. Below we get an overview including standard statistics for all numerical variables.

```
import pandas as pd
filename = "C:\\...\\tips.csv"
tips = pd.read_csv(filename)
tips.head(5)
```

Output:

```
   total_bill   tip     sex     smoker  day     time    size
0       16.99  1.01  Female    No      Sun     Dinner  2
1       10.34  1.66  Male      No      Sun     Dinner  3
2       21.01  3.50  Male      No      Sun     Dinner  3
3       23.68  3.31  Male      No      Sun     Dinner  2
4       24.59  3.61  Female    No      Sun     Dinner  4
```

```
tips.describe()
```

Output:

```
        total_bill       tip             size
count   244.000000       244.000000      244.000000
mean    19.785943        2.998279        2.569672
std     8.902412         1.383638        0.951100
min     3.070000         1.000000        1.000000
25%     13.347500        2.000000        2.000000
50%     17.795000        2.900000        2.000000
75%     24.127500        3.562500        3.000000
max     50.810000        10.000000       6.000000
```

```
tips["tip"].mean()  # automatically omits missing values
```

Output:

```
2.99827868852459
```

```
tips["tip"].std()
```

Output:

```
1.3836381890011826
```

```
tips["total_bill"].median()
```

Output:

```
17.795
```

```
tips.corr(numeric_only=True)
```

Output:

```
              total_bill       tip          size
total_bill      1.000000  0.675734      0.598315
tip             0.675734  1.000000      0.489299
size            0.598315  0.489299      1.000000
```

```
tips.corr(numeric_only=True, method = 'spearman')
```

Output:

```
              total_bill  tip               size
total_bill      1.000000  0.678968      0.604791
tip             0.678968  1.000000      0.468268
size            0.604791  0.468268      1.000000
```

```
tips.corr(numeric_only=True, method = 'kendall')
```

Output:

```
              total_bill       tip          size
total_bill      1.000000  0.517181      0.484342
tip             0.517181  1.000000      0.378185
size            0.484342  0.378185      1.000000
```

Data visualization with `matplotlib` & `seaborn`

`matplotlib`

Data visualisation is extremely important to understand your data and to share the insights you have acquired with others.

Clear graphics allow your data to tell their story and `matplotlib` makes it easy to create meaningful plots.

We will also use the subpackage *pyplot*, which is imported (by convention) as *plt*.

```
import matplotlib.pyplot as plt
import numpy as np
```

Histograms

Histograms are useful tools to explore your data, especially by allowing you to get an idea about the distribution of your variables.

You can use the hist function to create histograms in `matplotlib`.

More information about the function hist can be found in the help page: *help(plt.hist)*.

The first argument x should be the data, i.e. the list of values to build the histogram.

The second argument *bins* = tells Python into how many bins the data should be divided. The appropriate boundaries for all bins will be determined automatically. If you do not specify the number of bins, the default is 10.

```
values = [0.8, 1.4, 2.2, 2.6, 3.7, 4.1, 4.5, 5.1, 6.7, 7.2, 8.6,
   9.1]
plt.hist(values)
```

```
plt.hist(values, bins = 4, edgecolor="black")
```

We can improve the histogram by setting a more appropriate number of bins.

```
plt.hist(values, bins = [0, 2, 4, 6, 8, 10], edgecolor="black")
```

```
plt.hist(values, color="pink", bins = [0, 2, 4, 6, 8, 10],
   edgecolor="black")
```

Boxplots

```
plt.boxplot(values,showmeans=True)
plt.show()
```

Bar plot and pie chart

```
Affi=["KarU","KU","JKUAT","UON","KyU"]
Attend=[30,40,20,10,16]
plt.bar(Affi,Attend,color="pink",edgecolor="g")
```

```
plt.pie(Attend, labels=Affi)
```

```
extend=[0.2,0,0,0.1,0]
plt.pie(Attend, labels=Affi,explode=extend)
plt.show()
```

```
plt.pie(Attend, labels=Affi,explode=extend,autopct="%.2f%%")
plt.show()
```

Line charts

```
years = [2000, 2004, 2008, 2012, 2016]
prison_pop = [8.688, 9.245, 9.858, 11.212, 11.071]
```

```
plt.plot(years, prison_pop)
#         x-axis y-axis
```

```
plt.plot(years, prison_pop,c="g",lw=3)
```

```
plt.plot(years, prison_pop,c="g",lw=3, linestyle="--")
```

Scatterplots

```
plt.scatter(years, prison_pop)
```

You can see all the individual data points in this plot. Python does not connect them, which makes a scatterplot the best choice when plotting this type of data. Indeed, it is clear that the plot is based on only five data points.

```
plt.plot(years, prison_pop)
plt.scatter(years, prison_pop)
plt.show()
```

```
xdata=np.random.random(150)*100
delta=np.random.normal(2,10,150)
ydata=0.5+3*xdata +delta
plt.scatter(xdata, ydata,c="red")
```

```
plt.scatter(xdata, ydata,c="red", marker="+")
```

```
plt.scatter(xdata, ydata,c="red", marker=".",s=150)
```

Plot options

For each of these plots, you can do many customizations, such as changing colors, labels, and shapes.

Let us add some features to our previous simple scatterplot.

```
plt.scatter(years, prison_pop)
```

```
plt.scatter(years, prison_pop)
plt.xlabel("Years")      # label x-axis
plt.ylabel("Prison population")     # label y-axis
plt.title("Prison population in Belgium")    # add a title
plt.yticks([8.500, 9.000, 9.500, 10.000, 10.500, 11.00, 11.500])
    # set y-axis range from 8.500 to 11.500
```

Multilple plots in one figure

```
plt.figure(1)
plt.scatter(years, prison_pop)
plt.figure(2)
plt.pie(Attend, labels=Affi,explode=extend,autopct="%.2f%%")
```

```
fig,position=plt.subplots(2,2)
position[0,0].scatter(years, prison_pop)
position[0,0].set_title("scatter of of Pr pop")
position[0,1].plot(years, prison_pop)
position[0,1].set_title("evolution of Pr pop")
position[1,0].hist(prison_pop)
position[1,0].set_title("hist pop")
position[1,1].boxplot(prison_pop)
position[1,1].set_title("box pop")
fig.suptitle("Belgian Pris data")
plt.tight_layout()
```

Plotting with Seaborn and pandas

`matplotlib` works fine, but sometimes it is not enough. We might have more complicated data to plot, or we could simply want more sophisticated tools for data visualization.

To that end, we can use `Seaborn`, a powerful and easy to use Python library for creating common visualization types.

Note that, besides `Seaborn` (the easier `matplotlib`), we also need to import `matplotlib`, which is the library on top of which `Seaborn` is built.

```
import seaborn as sns   # sns is conventionally used as alias for
    Seaborn
import matplotlib.pyplot as plt
```

We want to create a scatter plot of the height and weight of 6 people. We compare the scatterplot function from `Seaborn` and the scatter function from `matplotlib` below. There is hardly any difference.

```
height = [1.55, 1.78, 1.65, 1.67, 1.88, 1.71]
weight = [52.1, 66.3, 65.8, 55.9, 84.5, 67.3]
sns.scatterplot(x = weight, y = height)   # use scatterplot
   function from Seaborn library
```

```
plt.scatter(weight, height)
```

Suppose we add a categorical variable like gender to our height and weight data. We could use a count plot to investigate how many observations are female or male.

```
gender = ["female", "female", "female", "female", "male", "male"]
     # categorical list gender
```

```
sns.countplot(x = gender)     # returns bars that represent the
   number of list entris per category
```

The simple plots we have made up til now, do not illustrate why `Seaborn` is so useful in data science.

`Seaborn` works extremely well with `pandas` data structure. We will use the `Seaborn` built-in `DataFrame` called tips as an example.

```
tips = sns.load_dataset("tips")
tips.head()
```

Each row represents a table served at a restaurant and has information about the bill, the tip, the customer's characteristics, timing and how many people were at the table.

Let us make a scatter plot of the total bill and tip columns.

```
sns.scatterplot(data = tips,
                x = "total_bill",
                y = "tip")
# sns.scatterplot(x = tips.total_bill, y = tips.tip) # same
   result
```

```
sns.scatterplot(x = tips.total_bill, y = tips.tip) # same result
```

Often customers include a percentage of the total bill as a tip. We already see that larger total bills are associated with larger tips.

Are there other variables that help explain the size of the tip? We might be interested in exploring the influence of gender. Below we use color (*hue*) to visualize men and women separately.

```
sns.scatterplot(data = tips,
                x = "total_bill",
                y = "tip",
                hue = "sex")
```

We can play around with the options. First we change the order in which the genders are colored.

```
sns.scatterplot(data = tips,
                x = "total_bill",
                y = "tip",
                hue = "sex",
                hue_order = ["Female", "Male"])
```

To change the color assigned to each gender, we can use the palette parameter. This parameter takes a dictionary (a data structure that has key - value pairs), which should map the variable values to the colors you want to represent them.

```
colors_dict = {"Female" : "blue",
               "Male" : "cyan"}

sns.scatterplot(data = tips,
                x = "total_bill",
                y = "tip",
                hue = "sex",
                hue_order = ["Female", "Male"],
                palette = colors_dict)
```

Have a look at some of the most common colors below.

Color	Matplotlib Name	Matplotlib Abbreviation
blue	"blue"	"b"
green	"green"	"g"
red	"red"	"r"
green/blue	"cyan"	"c"
purple	"magenta"	"m"
yellow	"yellow"	"y"
black	"black"	"k"
white	"white"	"w"

Visualizing two quantitative variables

In the last plot, we looked at the relationship between the total bill and tips at a restaurant. We highlighted two subgroups (male and female) with the hue parameter by using different color on the same plot.

There is another method to highlight subgroups: creating separate plots per subgroup. To do this, we need the Seaborn function *relplot*, which stands for **relational plots**.

We can use relplot to display two quantitative variable by using either scatter plots or line plots.

Scatter plots

One of the best features of *relplot* is the possibility to create subplots in a single figure. This is an example of how to use *relplot* using the tips data.

```
sns.relplot(data = tips,
            x = "total_bill",
            y = "tip",
```

```
            kind = "scatter")
```

```
sns.relplot(data = tips,
            x = "total_bill",
            y = "tip",
            kind = "scatter",       # kind parameter is used to
               specify which type of plots you want
            col = "sex", hue="smoker")            # obtain
               separate scatter plots for male and female,
               arranged horizontally in columns
```

```
sns.relplot(data=tips,
            x = "total_bill",
            y = "tip",
            kind = "scatter",
            col = "time",
            row = "day")
```

End of Lecture Notes — Happy Coding!